

T E C H N I S C H E D O K U M E N T A T I O N

Datenanalyse-Agent mit LangGraph

Stufe 2 – Reflexionsschleife mit austauschbaren Skills

Architektur-Review & Funktionsbeschreibung

Mai 2026

1 Überblick

Dieses Dokument beschreibt den Aufbau und die Funktionsweise der zweiten Ausbaustufe des agentischen Datenanalyse-Assistenten, implementiert als Jupyter Notebook. Gegenüber Stufe 1 (linearer Graph mit ReAct-Zyklus) führt Stufe 2 zwei wesentliche Erweiterungen ein:

- **Austauschbare Skills:** Analyse-Anweisungen werden nicht mehr als monolithischer String im Code hinterlegt, sondern als separate Markdown-Dateien verwaltet. Der Agent wählt zur Laufzeit basierend auf dem Datenprofil die passenden Skills aus und baut daraus seinen System-Prompt zusammen. Prompt-Engineering wird damit zur Konfiguration statt zur Codeänderung.
- **Reflexionsschleife (Reflection Pattern):** Ein separater Quality Checker bewertet die Analyseergebnisse gegen die aktiven Skills. Liegt der gewichtete Score unter einem konfigurierbaren Threshold (Standard: 70 %), wird die Analyse mit gezieltem Feedback zurück an den Analysten gegeben – maximal dreimal.

Technologie-Stack:

- LangGraph (Zustandsgraph),
- LangChain (LLM-Integration),
- Qwen/Nemotron über DeepInfra-API,
- Pandas, Matplotlib, Seaborn,
- Gradio (Web-UI).

2 Architektur

Der Agent ist als StateGraph mit integriertem ReAct-Zyklus und nachgelagerter Reflexionsschleife aufgebaut. Der Kontrollfluss folgt dem Muster:

```
START → data_profiler → analyst ⇄ tools → quality_checker → END
                                     ↳ analyst (max. 3×)
```

Gegenüber Stufe 1 kommen zwei neue Knoten hinzu: Der **quality_checker** bewertet die Analysequalität, und **increment_revision** zählt den Revisionsdurchlauf. Außerdem erhält der **data_profiler** die zusätzliche Aufgabe der Skill-Auswahl.

2.1 Knotenstruktur

Knoten	Typ	Aufgabe
data_profiler	Deterministisch	Datei einlesen, Struktur erfassen, Statistiken berechnen, passende Skills auswählen – kein LLM-Call
analyst	LLM + Tool-Calling	Analysen durchführen, gesteuert durch die geladenen Skills; bei Revision gezielt nachbessern
tools	ToolNode (Sandbox)	Python-Code sicher ausführen (pandas, matplotlib, seaborn, numpy)
quality_checker	LLM (günstiger)	Analyse gegen die aktiven Skills bewerten; gewichteten Score und strukturiertes Feedback erzeugen
increment_revision	Deterministisch	Revisionszähler um 1 erhöhen

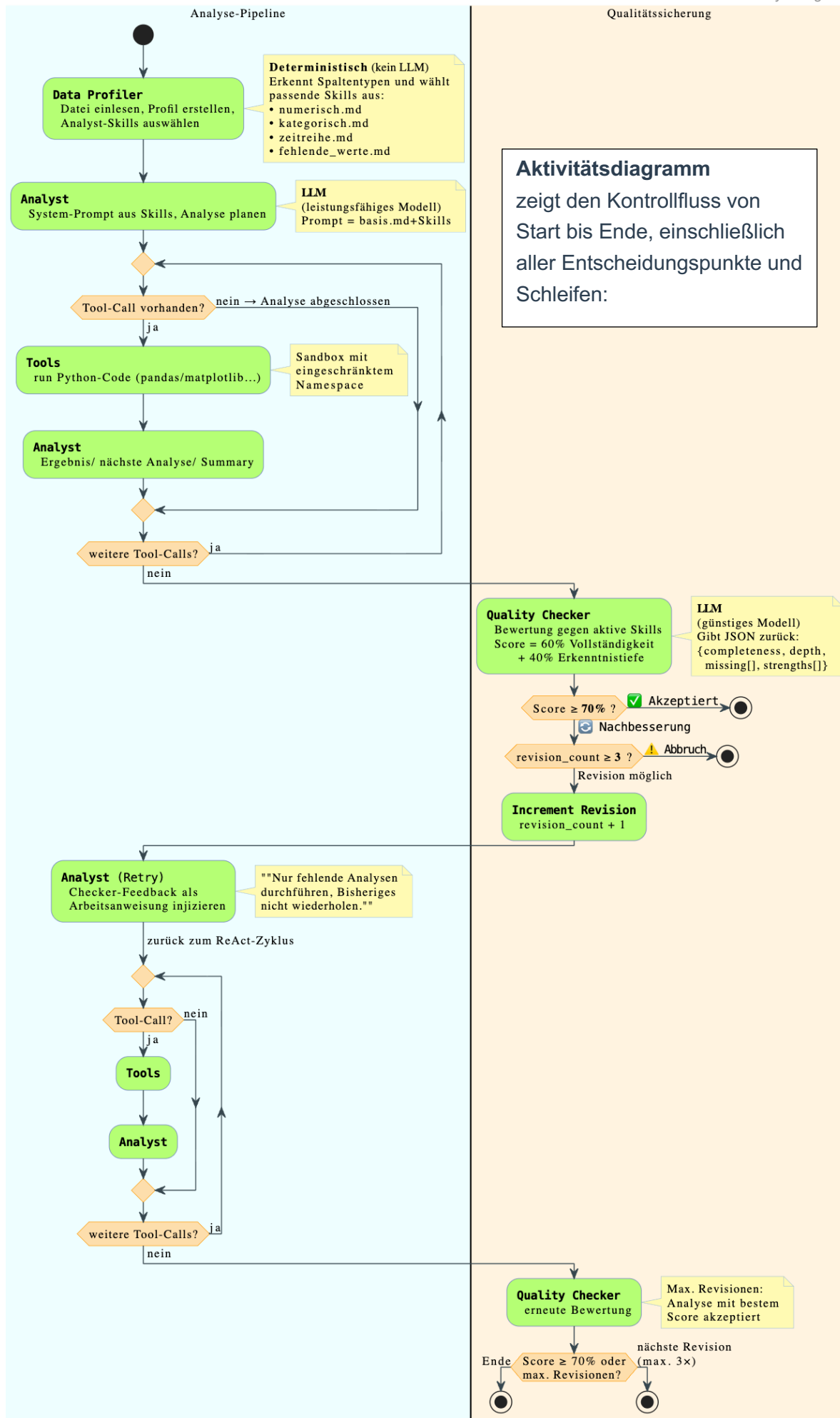
2.2 Kantenlogik

Der Graph verwendet zwei Kantentypen:

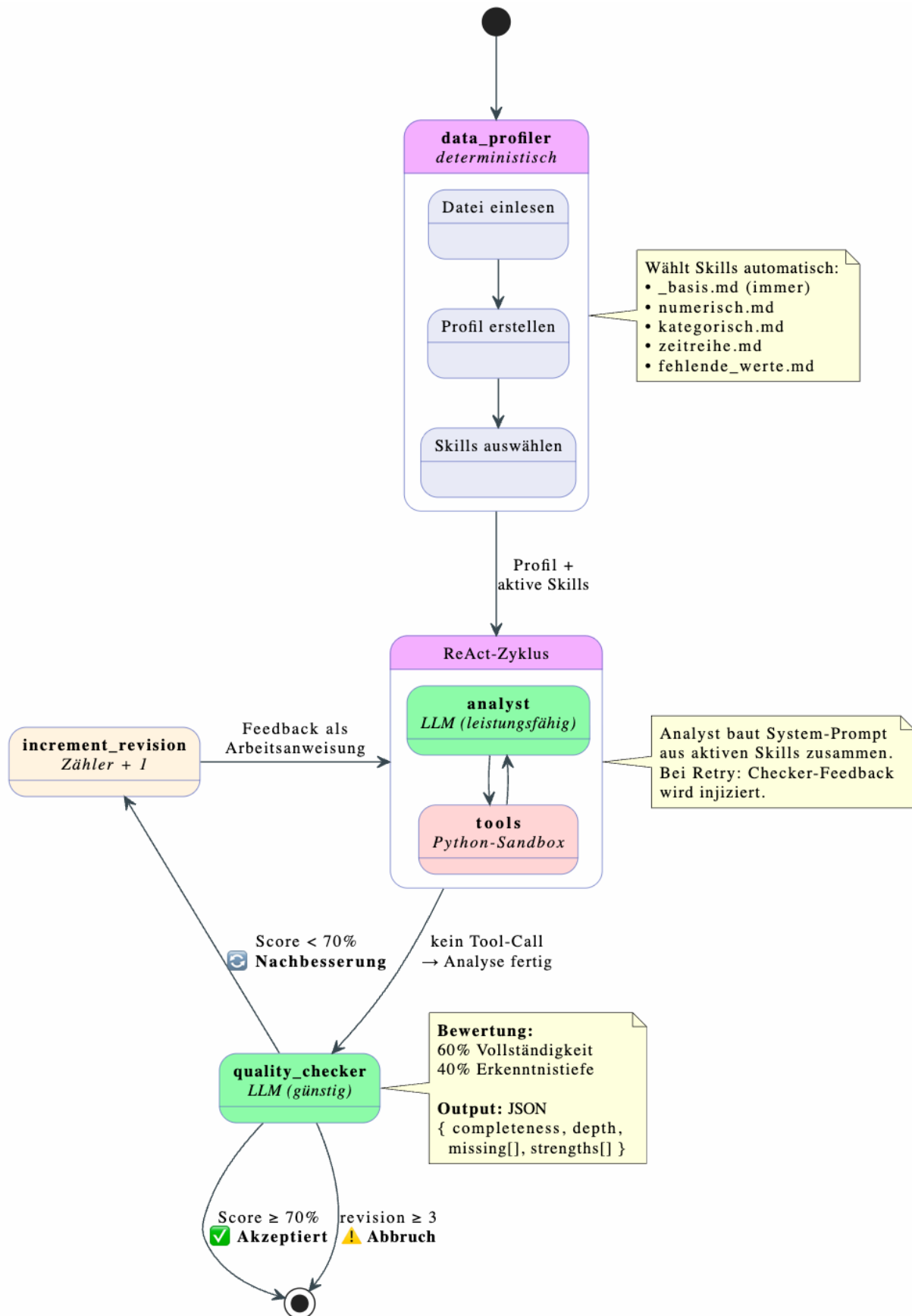
- **Feste Kanten** (add_edge): START → data_profiler → analyst, sowie tools → analyst und increment_revision → analyst (Rückführungen)
- **Konditionale Kante 1** (should_use_tool): Vom analyst-Knoten wird entschieden, ob der LLM einen Tool-Call gemacht hat (weiter zu tools) oder ob er fertig ist (weiter zu quality_checker)
- **Konditionale Kante 2** (should_revise): Vom quality_checker wird entschieden, ob der Score den Threshold erreicht (weiter zu END), oder ob eine Nachbesserung nötig ist (weiter zu increment_revision → analyst) – sofern das Maximum an Revisionen noch nicht erreicht ist

2.3 Diagramme

Die folgenden beiden PlantUML-Diagramme zeigen den Agenten aus zwei Perspektiven:



Zustandsdiagramm – zeigt die Knoten als Zustände und die Übergänge (feste und konditionale Kanten) zwischen ihnen:



3 Komponenten im Detail

3.1 State-Definition

Der State ist ein TypedDict mit sieben Feldern, das als zentrales Gedächtnis zwischen allen Knoten dient. Gegenüber Stufe 1 kommen vier neue Felder hinzu:

Feld	Typ	Beschreibung
file_path	str	Pfad zur hochgeladenen Datendatei
data_profile	dict	Strukturiertes Datenprofil aus dem Profiler-Knoten
active_skills	list[str]	Welche Analyst-Skills für diesen Durchlauf geladen sind (neu)
messages	list (Reducer)	LLM-Konversation mit add_messages-Reducer
quality_score	float	Gewichteter Qualitätsscore der letzten Bewertung (neu)
quality_feedback	str	Strukturiertes Feedback des Checkers (neu)
revision_count	int	Anzahl der bisherigen Nachbesserungsrunden (neu)

Der Reducer **add_messages** funktioniert identisch zu Stufe 1: Neue Nachrichten werden angehängt statt überschrieben, sodass die Konversationshistorie kontinuierlich wächst.

3.2 Tool: Python-Sandbox (execute_python)

Die Python-Sandbox ist identisch zu Stufe 1: Eine mit `@tool` dekorierte Funktion führt Python-Code in einem eingeschränkten Namespace aus (pandas, numpy, matplotlib, seaborn). Stdout und stderr werden abgefangen und als String zurückgegeben. Fehler werden gefangen und als Fehlertext zurückgegeben, damit der LLM seinen Code selbstständig korrigieren kann.

3.3 Data Profiler (mit Skill-Auswahl)

Der Data Profiler bleibt ein rein deterministischer Knoten ohne LLM-Beteiligung. Neben der bekannten Aufgabe aus Stufe 1 – Datei einlesen und Profil erstellen – übernimmt er jetzt zusätzlich die automatische Skill-Auswahl:

Die Funktion **select_analyst_skills** prüft das Datenprofil und wählt die passenden Analyse-Skills aus:

- Numerische Spalten vorhanden → numerisch.md wird geladen
- Kategorische Spalten vorhanden → kategorisch.md wird geladen
- Spaltennamen enthalten Datums-Keywords (datum, date, zeit, time, ...) → zeitreihe.md wird geladen
- Fehlende Werte vorhanden → fehlende_werte.md wird geladen

Die ausgewählten Skill-Namen werden im State-Feld **active_skills** gespeichert, damit der Quality Checker später weiß, wogegen er bewerten soll.

3.4 Skill-System

Das Skill-System ist die zentrale Neuerung gegenüber Stufe 1. Statt eines monolithischen System-Prompts werden Analyse-Anweisungen als separate Markdown-Dateien verwaltet:

```
skills/analyst/_basis.md      ← immer geladen (Grundregeln, Code-Konventionen)
skills/analyst/numerisch.md  ← bei numerischen Spalten
skills/analyst/kategorisch.md ← bei kategorischen Spalten
skills/analyst/zeitreihe.md   ← bei Datumsspalten
skills/analyst/fehlende_werte.md ← bei fehlenden Werten
skills/checker/standard.md    ← Bewertungskriterien für den Checker
```

Jede Skill-Datei kann **Platzhalter** enthalten, die zur Laufzeit mit echten Werten aus dem Datenprofil ersetzt werden:

Platzhalter	Wird ersetzt durch
{file_path}	Pfad zur Datendatei
{profile_json}	Vollständiges Datenprofil als JSON
{spalten}	Alle Spaltennamen (kommasepariert)
{numerische_spalten}	Nur numerische Spalten
{kategorische_spalten}	Nur kategorische Spalten
{fehlende_werte}	Dict der fehlenden Werte als JSON
{zeilen}	Anzahl Zeilen
{aktive_skills}	Liste der geladenen Skills (nur Checker)

Die Funktion **load_skill** lädt eine .md-Datei und ersetzt alle vorhandenen Platzhalter. Fehlende Platzhalter bleiben stehen, um Fehler durch unvollständige Kontexte zu vermeiden.

Die Funktion **build_analyst_prompt** setzt den finalen System-Prompt zusammen: zuerst _basis.md (immer), dann alle aktiven Spezial-Skills, verbunden durch Trennlinien. So entsteht ein modularer Prompt, dessen Bestandteile ohne Codeänderung angepasst oder erweitert werden können.

3.5 Analyst (Skill-gesteuert)

Der Analyst-Knoten ist weiterhin der Kern des Agenten mit dem ReAct-Zyklus (Reason + Act). Gegenüber Stufe 1 ändert sich sein Verhalten in zwei Punkten:

- **Skill-basierter System-Prompt:** Statt eines festen Strings wird der Prompt zur Laufzeit durch `build_analyst_prompt` aus den aktiven Skills zusammengebaut.
- **Revisions-Bewusstsein:** Bei Folgedurchläufen (`revision_count > 0`) wird das Checker-Feedback als `HumanMessage` injiziert. Der Analyst wird explizit angewiesen, nur die fehlenden Analysen durchzuführen und bereits Erledigtes nicht zu wiederholen.

Das LLM wird über **bind_tools** mit dem `execute_python`-Tool verbunden und entscheidet autonom: Tool-Call generieren (weiter zu tools) oder reine Textantwort (weiter zu `quality_checker`). Dies ist der gleiche ReAct-Zyklus wie in Stufe 1.

3.6 Quality Checker

Der Quality Checker ist der zweite neue LLM-Knoten und implementiert das Reflection Pattern. Er nutzt bewusst ein günstigeres Modell als der Analyst, da die Bewertungsaufgabe weniger komplex ist als die Analyse selbst.

Der Checker erhält seinen Prompt aus der Skill-Datei **skills/checker/standard.md** und bewertet die Analyse nach zwei Kriterien:

- **Vollständigkeit (completeness, 0.0–1.0):** Wurde jeder geladene Analyst-Skill abgearbeitet? Wenn z. B. `zeitreihe.md` aktiv war, aber der Analyst keine Zeitreihenanalyse durchgeführt hat, senkt das den Score.
- **Erkenntnistiefe (depth, 0.0–1.0):** Sind die Erkenntnisse spezifisch, mit Zahlen belegt, und gehen sie über Offensichtliches hinaus?

Aus beiden Einzelscores wird ein **gewichteter Gesamtscore** berechnet:

```
weighted_score = 0.60 × completeness + 0.40 × depth
```

Der Checker gibt ein strukturiertes JSON zurück mit den Feldern `completeness`, `depth`, `missing` (konkrete Lücken) und `strengths` (was gut war). Dieses Ergebnis wird robust geparkt – ein Regex extrahiert das JSON-Objekt aus der Antwort, und bei Parse-Fehlern greift ein Fallback mit neutralen Standardwerten.

3.7 Routing-Funktionen

Stufe 2 verwendet drei Routing-Funktionen:

- **should_use_tool:** Identisch zu Stufe 1 – prüft, ob die letzte LLM-Nachricht Tool-Calls enthält. Falls ja: „tools“, falls nein: „quality_checker“ (statt END in Stufe 1).
- **should_revise:** Neue Funktion. Prüft den gewichteten Score gegen den Threshold (Standard: 0.70). Falls der Score ausreicht oder das Maximum an Revisionen (Standard: 3) erreicht ist: END. Andernfalls: „analyst“ (via increment_revision).
- **increment_revision:** Deterministischer Hilfsknoten, der lediglich den Revisionszähler um 1 erhöht.

4 Ablauf einer Analyse

Ein vollständiger Durchlauf des Agenten umfasst folgende Schritte:

1. Der Nutzer lädt eine CSV- oder Excel-Datei hoch (via Gradio-Interface oder direkt als Pfad).
2. Der data_profiler liest die Datei ein, erstellt ein strukturiertes Profil und wählt basierend auf den erkannten Spaltentypen die passenden Analyst-Skills aus.
3. Der analyst erhält einen aus _basis.md und den aktiven Spezial-Skills zusammengesetzten System-Prompt. Er plant die erste Analyse und generiert einen Tool-Call mit Python-Code.
4. should_use_tool erkennt den Tool-Call und leitet zum tools-Knoten weiter.
5. Der ToolNode führt den Code aus und schreibt das Ergebnis als ToolMessage in den State.
6. Der analyst sieht das Ergebnis, plant weitere Analysen oder generiert neuen Code.
7. Schritte 4–6 wiederholen sich (typisch 2–4 Zyklen), bis der Analyst mit reinem Text antwortet.
8. should_use_tool erkennt das Fehlen von Tool-Calls und routet zum quality_checker.
9. Der quality_checker bewertet die Analyse gegen die aktiven Skills und berechnet den gewichteten Score.
10. should_revise prüft den Score: Liegt er über dem Threshold (70 %), wird zu END geroutet. Liegt er darunter, wird über increment_revision zurück zum Analysten geleitet.
11. Bei einer Revision erhält der Analyst das Checker-Feedback und führt gezielt die fehlenden Analysen durch (Schritte 3–10 wiederholen sich, max. 3×).
12. Der finale Markdown-Bericht wird im Gradio-Interface angezeigt.

5 Streaming & Berichterstellung

Die Funktion **run_analysis** nutzt `app.stream()` mit `stream_mode="updates"`, um bei jedem Graphschritt ein Dictionary zu erhalten. Gegenüber Stufe 1 werden nun zusätzliche Knotentypen verarbeitet:

- **data_profiler**: Zeilen-/Spaltenanzahl, Spaltentypen, fehlende Werte und die ausgewählten Skills
- **analyst**: Code-Previews der Tool-Calls oder die finale Zusammenfassung
- **tools**: Ausgabe-Previews der Code-Ausführung
- **quality_checker**: Gewichteter Score und das strukturierte Feedback (neu)
- **increment_revision**: Revisionsnummer und Hinweis auf Nachbesserung (neu)

Die Funktion ist als Generator implementiert und gibt nach jedem Schritt den aktuellen Zwischenbericht zurück, sodass Gradio das Interface live aktualisieren kann. Der finale Bericht gliedert sich in vier Abschnitte: Analyseverlauf, Qualitätsbewertung, Ergebnisse und Erkenntnisse sowie der verwendete Analyse-Code.

6 Benutzeroberfläche (Gradio)

Das Gradio-Interface ist funktional identisch zu Stufe 1, wurde aber in der Beschreibung um die neuen Konzepte ergänzt. Es zeigt den aktuellen Threshold und die maximale Revisionszahl an. Der Datei-Upload akzeptiert CSV-, Excel- und TSV-Dateien, und die vorbereitete Beispieldatei mit 30 Zeilen Verkaufsdaten (inklusive zweier Datensätze mit absichtlich fehlenden Werten) dient als Schnellstart.

Durch die Generator-basierte Implementierung von `run_analysis` aktualisiert Gradio die Markdown-Ausgabe nach jedem Graphschritt – einschließlich der Quality-Check-Ergebnisse und etwaiger Revisionsrunden.

7 LLM-Konfiguration

Stufe 2 verwendet bewusst **zwei verschiedene Modelle** über die DeepInfra-API:

Rolle	Modell	Konfiguration
Analyst	Qwen3.6-35B-A3B	temperature=0, max_tokens=5000
Checker	Nemotron-3-Nano-Omni-30B-A3B	temperature=0, max_tokens=2000

Die Trennung in zwei Modelle ist eine bewusste Architekturentscheidung: Der Analyst benötigt ein leistungsfähiges Modell für komplexe Analyse und Code-Generierung. Der Checker muss lediglich eine Bewertung als JSON-Objekt abgeben – eine einfachere Aufgabe, für die ein günstigeres Modell ausreicht. Dies senkt die Gesamtkosten pro Analysedurchlauf.

Beide Modelle werden über die ChatOpenAI-Klasse von LangChain angebunden, wobei der API-Endpunkt auf DeepInfra umgeleitet wird. Matplotlib wird auf das Agg-Backend gesetzt, damit Plots als Dateien gespeichert statt in einem GUI-Fenster angezeigt werden.

8 Konfigurationsparameter

Alle konfigurierbaren Parameter sind an einer zentralen Stelle im Notebook definiert:

Parameter	Standard	Beschreibung
QUALITY_THRESHOLD	0.70	Mindest-Score für Akzeptanz der Analyse
MAX_REVISIONS	3	Maximale Anzahl Nachbesserungsrunden
WEIGHT_COMPLETENESS	0.60	Gewicht der Vollständigkeit im Score
WEIGHT_DEPTH	0.40	Gewicht der Erkenntnistiefe im Score
SKILLS_DIR	skills/	Pfad zum Skills-Verzeichnis (relativ zum Notebook)

9 Erweiterbarkeit: Eigene Skills erstellen

Das Skill-System ist bewusst so gestaltet, dass neue Analyse-Skills ohne Codeänderung hinzugefügt werden können. Der Prozess umfasst drei Schritte:

1. **Markdown-Datei anlegen:** Eine neue .md-Datei in skills/analyst/ erstellen, die Analyse-Anweisungen und Platzhalter enthält (z. B. ecommerce.md für E-Commerce-spezifische Analysen).
2. **Skill-Auswahl anpassen:** In der Funktion select_analyst_skills eine neue Bedingung ergänzen, die den Skill bei passenden Spaltennamen oder Datentypen aktiviert.
3. **Optional – Checker-Skill erweitern:** Die Bewertungskriterien in skills/checker/standard.md um den neuen Skill-Bereich ergänzen, damit der Quality Checker auch die domänenspezifischen Analysen bewerten kann.

Der Basis-Skill _basis.md (mit führendem Unterstrich) wird immer geladen und enthält die grundlegenden Code-Konventionen und Regeln. Alle weiteren Skills werden vom Data Profiler basierend auf dem Datenprofil ausgewählt.